ANGULAR TALENT LAB 2026

Devoir pratique — Week-end intensif

Le Calao Doré

Site vitrine d'un restaurant camerounais

🗓️ **Rendu :** lundi 22 juin 2026, avant 9h00

⏱️ **Durée :** samedi 20 → dimanche 21 juin (week-end)

👤 **Mode :** Solo — projet personnel sur votre GitHub

🎯 **Note :** 20 points (grille détaillée en fin de document)

Le contexte — votre première mission freelance

Vous venez d'être contacté(e) par Madame Ngono, propriétaire d'un restaurant traditionnel à Douala, quartier Akwa. Son établissement, Le Calao Doré, sert depuis 15 ans une cuisine camerounaise authentique (Ndolè, Eru, Poulet DG, Mbongo Tchobi...) mais elle n'a aucune présence en ligne.

Elle vous demande de créer le site vitrine de son restaurant. Elle a un budget limité mais une exigence claire : que son site soit moderne, joli sur téléphone (sa clientèle consulte surtout depuis son mobile), et qu'il donne envie de venir manger.

Vous êtes seul(e) sur ce projet. Vous avez le week-end. À vous de jouer.

OBJECTIF PÉDAGOGIQUE

Ce devoir mobilise TOUS les acquis des 4 premiers jours dans un contexte concret :

- Standalone Components (J2) : architecture en composants imbriqués
- Layout responsive (J3) : Desktop / Tablette / Mobile
- Bootstrap (J4) : grilles et composants UI
- Control flow @if / @for / @switch (J4)
- Styles dynamiques [class.X], ngClass, [style.X], ngStyle (J4)

Le résultat sera votre PREMIER projet utilisable sur votre portfolio professionnel.

PERSONNALISATION ENCOURAGÉE

Madame Ngono est un personnage fictif. Vous pouvez :

- Garder Le Calao Doré et le quartier d'Akwa (option par défaut)
- OU inventer votre propre restaurant camerounais (nom, quartier, plats)
- L'IMPORTANT : que ce soit cohérent, camerounais, et que tous les contenus soient en français.

Plus votre site sera personnel et authentique, plus il vous servira au moment de chercher un job.

Les livrables attendus

À 9h00 le lundi 22 juin, vous devez avoir publié sur GitHub :

1. Un repository public nommé exactement : le-calao-dore (ou le nom de votre restaurant en kebab-case).
2. Le code source complet de votre application Angular 22.
3. Un fichier README.md à la racine avec : nom du projet, votre nom, captures d'écran (Desktop + Mobile), commande pour lancer le projet.
4. Un historique Git propre avec au minimum 4 commits significatifs (un par bloc de travail).
5. L'invitation envoyée à ebangaarnaud@gmail.com comme collaborateur du repo.

⚠ TOUT REPO REMIS APRÈS 9H00 LUNDI = PÉNALITÉ DE 4 POINTS

Le rendu en temps voulu fait partie de l'évaluation. C'est une compétence pro fondamentale.

Si vous avez un blocage majeur (panne PC, internet HS), prévenez sur le WhatsApp de la cohorte AVANT 9h00.

Mieux vaut rendre un projet partiellement fini que ne rien rendre du tout.

La maquette à reproduire

Voici la structure visuelle attendue, en vue Desktop. Vous adapterez ensuite pour Tablette et Mobile (consignes plus loin).

Vue Desktop (≥ 900px)

HEADER — logo + navbar (Accueil, Menu, Témoignages, Horaires, Contact)

HERO — image de fond + titre accroche + bouton CTA "Voir le menu"

SECTION MENU — titre + 3 colonnes (Entrées / Plats / Desserts)

Chaque plat = card Bootstrap avec nom, prix, badge catégorie

SECTION TÉMOIGNAGES — 3 cards horizontales avec étoiles

SECTION HORAIRES — 7 jours de la semaine + statut OUVERT/FERMÉ dynamique

FOOTER — contact + réseaux sociaux + copyright

Palette de couleurs imposée

Rôle	Nom	Code hexa	Aperçu
Vert foncé	Forêt camerounaise	#2D4A3E	
Terracotta	Terre rouge	#C84B31	
Or	Soleil tropical	#E9B824	
Crème	Sable Kribi	#FAF3E0	

Ces couleurs sont obligatoires pour conserver l'identité visuelle camerounaise. Vous pouvez ajouter quelques nuances complémentaires (gris pour le texte, blanc pour le fond de certaines sections) mais gardez ces 4 couleurs comme base.

Les 3 vues responsive attendues

Desktop (≥ 900px)

Conformément à la maquette ci-dessus. Header avec navbar horizontale, sections empilées verticalement, menu en 3 colonnes, témoignages en 3 cards horizontales.

Tablette (600 – 900px)

Header reste identique mais la navbar peut se compacter (espacement réduit entre les liens). La section Menu passe de 3 colonnes à 2 colonnes. Les témoignages restent en 3 cards mais avec moins de padding.

Mobile (< 600px)

- La navbar disparaît visuellement et est remplacée par une icône burger (3 traits horizontaux) à droite du logo.
- Quand on clique sur l'icône burger, le menu s'affiche en colonne sous le header.
- La section Menu passe en 1 colonne.
- Les témoignages s'empilent verticalement.
- Tous les boutons sont assez gros pour le doigt (min 44x44 pixels).

MOBILE-FRIENDLY = OBLIGATOIRE

La cible principale du Calao Doré est mobile. Madame Ngono a insisté.

Avant de rendre votre projet, OUVREZ-LE sur votre vrai téléphone (depuis le WiFi de votre PC) et vérifiez que :

• Tout est lisible sans zoomer

• Les boutons sont cliquables facilement avec le pouce

• Le menu burger fonctionne

• L'image hero ne déborde pas

Si ça marche sur votre téléphone, ça marchera pour Madame Ngono.

Section 1 — Header avec navbar responsive

Ce qu'il faut faire

Créer un composant <app-header> contenant :

- Le logo (texte "Le Calao Doré" en or sur fond vert foncé, police bold)
- Une navbar avec 5 liens : Accueil, Menu, Témoignages, Horaires, Contact
- Sur mobile : un bouton burger qui affiche/cache la navbar

Code TypeScript du composant

📄 **header.component.ts**

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-header',
  imports: [],
  templateUrl: './header.component.html',
  styleUrl: './header.component.css'
})
export class HeaderComponent {
  isMenuOpen = false;
  activeLink = 'Accueil';

  links = ['Accueil', 'Menu', 'Témoignages', 'Horaires', 'Contact'];

  toggleMenu() {
    this.isMenuOpen = !this.isMenuOpen;
  }

  setActive(link: string) {
    this.activeLink = link;
    this.isMenuOpen = false; // ferme le menu mobile après clic
  }
}
```

Code HTML du template

📄 **header.component.html**

```html
<header>
  <div class="logo">Le Calao Doré</div>

  <!-- Bouton burger visible uniquement sur mobile -->
```

```
  <button class="burger" (click)="toggleMenu()">☰</button>

  <!-- Navbar : visible par défaut sur desktop, conditionnelle sur mobile -->
  @if (isMenuOpen || !isMobile) {
    <nav>
      @for (link of links; track link) {
        <a [class.active]="activeLink === link"
           (click)="setActive(link)">
          {{ link }}
        </a>
      }
    </nav>
  }
</header>
```

🎯 POINTS PÉDAGOGIQUES MOBILISÉS

- Standalone Component avec imports: [] (J2)
- @for avec track (J4)
- @if pour le toggle burger mobile (J4)
- [class.active] pour le lien actif (J4)
- (click) événement DOM (J4)

Cette section seule mobilise déjà 5 acquis. Faites-la bien.

Section 2 — Hero avec image de fond

Ce qu'il faut faire

Créer un composant <app-hero> contenant :

- Une grande image de fond (au choix : nourriture camerounaise, terrasse de restaurant, plat traditionnel)
- Un overlay sombre semi-transparent pour rendre le texte lisible
- Un titre d'accroche : "Saveurs authentiques du Cameroun"
- Un sous-titre : "15 ans de tradition culinaire à Douala"
- Un bouton CTA : "Voir le menu" qui scrolle vers la section Menu

Astuce — utiliser [style.X] pour l'image de fond

L'image de fond doit être dynamique pour pouvoir changer facilement. Stockez son URL dans une variable du composant :

hero.component.ts

```typescript
export class HeroComponent {
  backgroundImage = '/assets/images/hero-restaurant.jpg';
  title = 'Saveurs authentiques du Cameroun';
  subtitle = '15 ans de tradition culinaire à Douala';
}
```

hero.component.html

```html
<section class="hero"
         [style.background-image]="'url(' + backgroundImage + ')'">
  <div class="overlay"></div>
  <div class="content">
    <h1>{{ title }}</h1>
    <p>{{ subtitle }}</p>
    <a href="#menu" class="btn btn-warning btn-lg">Voir le menu</a>
  </div>
</section>
```

OÙ TROUVER DES IMAGES LIBRES DE DROITS

Vous DEVEZ utiliser des images libres de droits. Quelques sources :

• unsplash.com — recherchez « cameroonian food », « african restaurant »

• pexels.com — idem

• Vos propres photos prises dans un vrai restaurant local

Téléchargez les images dans src/assets/images/ de votre projet.

INTERDIT : prendre une image trouvée sur Google sans vérifier les droits.

Section 3 — Le menu du restaurant

Ce qu'il faut faire

Créer un composant <app-menu> qui affiche les plats du restaurant en cards Bootstrap. Chaque plat doit avoir :

- Un nom
- Une description courte
- Un prix en FCFA
- Une catégorie : entrée, plat ou dessert
- Un badge coloré qui change selon la catégorie

Données à utiliser (interface TypeScript)

📄 **menu.component.ts**

TypeScript

```
import { Component } from '@angular/core';

interface Dish {
  id: number;
  name: string;
  description: string;
  price: number;
  category: 'entree' | 'plat' | 'dessert';
}

@Component({
  selector: 'app-menu',
  imports: [],
  templateUrl: './menu.component.html',
  styleUrl: './menu.component.css'
})
export class MenuComponent {
  dishes: Dish[] = [
    { id: 1, name: 'Beignets haricots', description: 'Beignets de niébé maison',
      price: 1500, category: 'entree' },
    { id: 2, name: 'Folong', description: 'Salade traditionnelle au folon',
      price: 2000, category: 'entree' },
    { id: 3, name: 'Ndolè aux crevettes', description: 'Spécialité douala
emblématique',
      price: 5500, category: 'plat' },
    { id: 4, name: 'Poulet DG', description: 'Plantain mûr, légumes croquants',
      price: 6000, category: 'plat' },
    { id: 5, name: 'Eru aux écailles', description: 'Légume forestier, viande fumée',
      price: 5000, category: 'plat' },
    { id: 6, name: 'Mbongo Tchobi', description: 'Sauce noire aux épices sauvages',
      price: 5500, category: 'plat' },
```

```
    { id: 7, name: 'Pain perdu coco', description: 'Brioche, lait de coco, miel',
      price: 2500, category: 'dessert' },
    { id: 8, name: 'Bananes flambées', description: 'Plantain caramélisé, rhum',
      price: 2000, category: 'dessert' },
  ];
}
```

Code HTML du template

📄 **menu.component.html**

HTML

```
<section id="menu" class="container py-5">
  <h2 class="text-center mb-5">Notre menu</h2>

  <div class="row">
    @for (dish of dishes; track dish.id) {
      <div class="col-12 col-md-6 col-lg-4 mb-4">
        <div class="card h-100">
          <div class="card-body">
            <!-- Badge dynamique selon la catégorie -->
            <span class="badge"
                  [ngClass]="{
                    'bg-success': dish.category === 'entree',
                    'bg-danger': dish.category === 'plat',
                    'bg-warning text-dark': dish.category === 'dessert'
                  }">
              @switch (dish.category) {
                @case ('entree') { Entrée }
                @case ('plat') { Plat }
                @case ('dessert') { Dessert }
              }
            </span>

            <h5 class="card-title mt-2">{{ dish.name }}</h5>
            <p class="card-text text-muted">{{ dish.description }}</p>
            <p class="price fw-bold">{{ dish.price }} FCFA</p>
          </div>
        </div>
      </div>
    }
  </div>
</section>
```

🎯 POINTS PÉDAGOGIQUES MOBILISÉS

- @for avec track sur un id (best practice J4)
- @switch / @case pour le label du badge (J4)

- [ngClass] avec 3 conditions exclusives (J4)
- Grille Bootstrap responsive : col-12 (mobile) / col-md-6 (tablette) / col-lg-4 (desktop)
- Interpolation {{ }} pour afficher les données

C'est la section la plus DENSE pédagogiquement. Faites-la avec soin.

Section 4 — Témoignages clients

Ce qu'il faut faire

Créer un composant <app-temoignages> qui affiche 3 témoignages de clients fictifs (mais réalistes !). Chaque témoignage doit avoir :

- Un nom du client (camerounais)
- Une note de 1 à 5 étoiles
- Un texte de témoignage
- Une couleur de fond qui change selon la note (rouge si < 3, jaune si = 3, vert si > 3)

Données à utiliser

📄 **temoignages.component.ts**

TypeScript

```
interface Testimonial {
  id: number;
  name: string;
  rating: number;
  text: string;
}

export class TemoignagesComponent {
  testimonials: Testimonial[] = [
    {
      id: 1,
      name: 'Aïssatou M.',
      rating: 5,
      text: 'Le Ndolè est le meilleur de Douala. Service impeccable.'
    },
    {
      id: 2,
      name: 'Jean-Pierre K.',
      rating: 4,
      text: 'Ambiance chaleureuse et plats généreux. Je recommande.'
    },
    {
      id: 3,
      name: 'Christelle E.',
      rating: 5,
      text: 'Cadre magnifique, cuisine authentique, prix raisonnables.'
    }
  ];

  // Helper : génère un tableau de la longueur de la note
  // pour afficher N étoiles avec @for
  getStars(rating: number): number[] {
```

```
    return Array(rating).fill(0);
  }
}
```

Code HTML du template

📄 **temoignages.component.html**

HTML

```
<section class="testimonials-section py-5">
  <div class="container">
    <h2 class="text-center mb-5">Ce qu'ils en disent</h2>

    <div class="row">
      @for (t of testimonials; track t.id) {
        <div class="col-12 col-md-4 mb-3">
          <div class="testimonial-card p-4"
               [class.bg-success]="t.rating > 3"
               [class.bg-warning]="t.rating === 3"
               [class.bg-danger]="t.rating < 3">

            <!-- Affichage des étoiles avec @for imbriqué -->
            <div class="stars mb-2">
              @for (star of getStars(t.rating); track $index) {
                <span>⭐</span>
              }
            </div>

            <p class="testimonial-text">{{ t.text }}</p>
            <p class="testimonial-author fw-bold">— {{ t.name }}</p>
          </div>
        </div>
      }
    </div>
  </div>
</section>
```

ASTUCE — @for imbriqué pour les étoiles

Un @for à l'intérieur d'un @for ! Le premier boucle sur les témoignages, le deuxième boucle sur le nombre d'étoiles.

Pour le track du @for imbriqué, on utilise $index car les étoiles sont identiques entre elles.

C'est une technique très courante dans les apps réelles.

Section 5 — Horaires avec statut dynamique

Ce qu'il faut faire

Créer un composant <app-horaires> qui affiche les jours de la semaine avec leurs horaires. EN PLUS : afficher un badge "OUVERT MAINTENANT" si on est dans les horaires d'ouverture, sinon "FERMÉ" en rouge.

Données à utiliser

📄 **horaires.component.ts**

TypeScript

```
interface Horaire {
  jour: string;
  ouverture: string;
  fermeture: string;
  ferme: boolean;
}

export class HorairesComponent {
  horaires: Horaire[] = [
    { jour: 'Lundi',    ouverture: '11h00', fermeture: '22h00', ferme: false },
    { jour: 'Mardi',    ouverture: '11h00', fermeture: '22h00', ferme: false },
    { jour: 'Mercredi', ouverture: '11h00', fermeture: '22h00', ferme: false },
    { jour: 'Jeudi',    ouverture: '11h00', fermeture: '22h00', ferme: false },
    { jour: 'Vendredi', ouverture: '11h00', fermeture: '23h00', ferme: false },
    { jour: 'Samedi',   ouverture: '11h00', fermeture: '23h00', ferme: false },
    { jour: 'Dimanche', ouverture: '',      fermeture: '',      ferme: true  }
  ];

  // Helper : vrai jour courant en français
  joursFR = ['Dimanche', 'Lundi', 'Mardi', 'Mercredi',
             'Jeudi', 'Vendredi', 'Samedi'];

  get jourActuel(): string {
    return this.joursFR[new Date().getDay()];
  }
}
```

Code HTML du template

📄 **horaires.component.html**

HTML

```
<section class="horaires-section py-5">
  <div class="container">
    <h2 class="text-center mb-5">Nos horaires d'ouverture</h2>
```

```
    <ul class="list-group">
      @for (h of horaires; track h.jour) {
        <li class="list-group-item d-flex justify-content-between
                   align-items-center"
            [class.current-day]="h.jour === jourActuel">

          <span class="fw-bold">{{ h.jour }}</span>

          @if (h.ferme) {
            <span class="badge bg-danger">Fermé</span>
          } @else {
            <span>
              {{ h.ouverture }} - {{ h.fermeture }}
              @if (h.jour === jourActuel) {
                <span class="badge bg-success ms-2">
                  Ouvert aujourd'hui
                </span>
              }
            </span>
          }
        </li>
      }
    </ul>
  </div>
</section>
```

🎯 LA DIFFICULTÉ DE CETTE SECTION

Cette section combine PLUSIEURS concepts à la fois :

- @for sur les jours de la semaine
- @if / @else pour différencier jour fermé vs jour ouvert
- @if imbriqué pour le badge "Ouvert aujourd'hui"
- [class.current-day] pour mettre en évidence le jour actuel
- Getter TypeScript jourActuel qui utilise l'API Date du navigateur

C'est ici qu'on sépare ceux qui ont vraiment compris des autres.

Section 6 — Footer responsive

Ce qu'il faut faire

Créer un composant <app-footer> contenant 3 colonnes :

- Colonne 1 : Coordonnées (adresse à Akwa, téléphone, email)
- Colonne 2 : Réseaux sociaux (Facebook, Instagram, WhatsApp) avec icônes
- Colonne 3 : Liens rapides (Menu, Horaires, Contact)

Sur mobile : les 3 colonnes s'empilent verticalement.

En bas du footer : copyright "© 2026 Le Calao Doré — Tous droits réservés"

Astuce — utiliser Bootstrap

📄 **footer.component.html**

HTML

```
<footer class="footer py-4">
  <div class="container">
    <div class="row">
      <!-- Colonne 1 : Coordonnées -->
      <div class="col-12 col-md-4">
        <h5>Le Calao Doré</h5>
        <p>📍 Rue Soudanaise, Akwa - Douala</p>
        <p>📞 +237 6XX XX XX XX</p>
        <p>✉ contact@calao-dore.cm</p>
      </div>

      <!-- Colonne 2 : Réseaux sociaux -->
      <div class="col-12 col-md-4">
        <h5>Suivez-nous</h5>
        <div class="socials">
          <a href="#">Facebook</a>
          <a href="#">Instagram</a>
          <a href="#">WhatsApp</a>
        </div>
      </div>

      <!-- Colonne 3 : Liens rapides -->
      <div class="col-12 col-md-4">
        <h5>Navigation</h5>
        <ul class="list-unstyled">
          <li><a href="#menu">Menu</a></li>
          <li><a href="#horaires">Horaires</a></li>
          <li><a href="#contact">Contact</a></li>
        </ul>
      </div>
    </div>
```

```
    <hr>
    <p class="text-center mb-0">
      © 2026 Le Calao Doré — Tous droits réservés
    </p>
  </div>
</footer>
```

La hiérarchie de composants attendue

Voici la structure de fichiers que vous devez créer dans src/app/components/ :

Arborescence

```
src/app/
├── app.ts                          (composant racine)
├── app.html
├── app.config.ts
├── components/
│   ├── header/                     (Section 1)
│   │   ├── header.component.ts
│   │   ├── header.component.html
│   │   └── header.component.css
│   ├── hero/                       (Section 2)
│   │   ├── hero.component.ts
│   │   ├── hero.component.html
│   │   └── hero.component.css
│   ├── menu/                       (Section 3)
│   │   ├── menu.component.ts
│   │   ├── menu.component.html
│   │   └── menu.component.css
│   ├── temoignages/                (Section 4)
│   │   ├── temoignages.component.ts
│   │   ├── temoignages.component.html
│   │   └── temoignages.component.css
│   ├── horaires/                   (Section 5)
│   │   ├── horaires.component.ts
│   │   ├── horaires.component.html
│   │   └── horaires.component.css
│   └── footer/                     (Section 6)
│       ├── footer.component.ts
│       ├── footer.component.html
│       └── footer.component.css
└── assets/
    └── images/
        └── hero-restaurant.jpg
```

Et app.html assemble simplement tous ces composants :

app.html

HTML

```
<app-header></app-header>
<app-hero></app-hero>
<app-menu></app-menu>
<app-temoignages></app-temoignages>
<app-horaires></app-horaires>
<app-footer></app-footer>
```

N'oubliez pas d'importer tous ces composants dans app.ts !

Roadmap suggérée du week-end

Voici comment je vous propose de découper votre travail. Adaptez selon votre rythme, mais respectez l'ordre — chaque section dépend de la précédente.

Samedi matin (8h00 — 12h00) — Setup + Header + Hero

- ☐ 8h00 — 8h30 : Créer le projet Angular 22 + initialiser Git
- ☐ 8h30 — 9h00 : Installer Bootstrap avec ng add bootstrap
- ☐ 9h00 — 9h15 : Créer le repo GitHub + premier commit (setup vide)
- ☐ 9h15 — 10h30 : Section 1 — Header avec navbar responsive
- ☐ 10h30 — 10h45 : PAUSE
- ☐ 10h45 — 12h00 : Section 2 — Hero avec image de fond

Samedi après-midi (14h00 — 18h00) — Menu + Témoignages

- ☐ 14h00 — 16h00 : Section 3 — Le menu (la plus dense, prenez le temps)
- ☐ 16h00 — 16h15 : PAUSE
- ☐ 16h15 — 18h00 : Section 4 — Témoignages
- ☐ 18h00 : COMMIT "Sections 1 à 4 terminées"

Dimanche matin (8h00 — 12h00) — Horaires + Footer + Responsive

- ☐ 8h00 — 9h30 : Section 5 — Horaires avec statut dynamique
- ☐ 9h30 — 9h45 : PAUSE
- ☐ 9h45 — 11h00 : Section 6 — Footer
- ☐ 11h00 — 12h00 : Vérification responsive Desktop / Tablette / Mobile

Dimanche après-midi (14h00 — 18h00) — Finitions + README + Tests

- ☐ 14h00 — 15h00 : Ajustements visuels (espacements, polices)
- ☐ 15h00 — 16h00 : Tester sur VRAI téléphone via WiFi
- ☐ 16h00 — 17h00 : Écrire le README.md (description + screenshots)
- ☐ 17h00 — 18h00 : Commit final + push GitHub + invitation collaborateur

Dimanche soir (19h00 — 21h00) — BUFFER (au cas où)

Gardez 2h de marge le dimanche soir pour les imprévus : un bug qui résiste, une dernière retouche, une image à changer. Ne planifiez RIEN d'autre.

> **STRATÉGIE ANTI-STRESS**

> Si vous prenez du retard samedi : laissez tomber les finitions visuelles, concentrez-vous sur le FONCTIONNEL.
>
> Mieux vaut un site moche mais qui marche, qu'un site beau mais cassé.
>
> Les recruteurs jugent d'abord la qualité du code, ensuite seulement l'apparence.

Workflow Git attendu

Votre historique Git doit montrer une progression cohérente. Voici la liste minimale de commits attendus :

6. Initial commit — projet Angular vide créé
7. feat: ajout Bootstrap via ng add
8. feat(header): header avec navbar responsive et menu burger
9. feat(hero): section hero avec image de fond dynamique
10. feat(menu): section menu avec @for + @switch + ngClass
11. feat(temoignages): section témoignages avec étoiles dynamiques
12. feat(horaires): section horaires avec statut jour actuel
13. feat(footer): footer responsive 3 colonnes
14. style: ajustements responsive Desktop/Tablette/Mobile
15. docs: ajout README.md avec captures d'écran

Exemples de bons messages de commit

❌ MAUVAIS	☑ BON
`"update"`	`"feat(header): ajout menu burger mobile"`
`"fix bug"`	`"fix(menu): badge dessert maintenant en jaune"`
`"css"`	`"style(footer): empilement vertical sur mobile"`
`"commit"`	`"feat(horaires): badge dynamique selon jour actuel"`

LA CONVENTION CONVENTIONAL COMMITS

feat: nouvelle fonctionnalité
fix: correction de bug
style: changement de CSS / mise en forme
docs: documentation (README, commentaires)
refactor: réorganisation du code sans changement fonctionnel
Cette convention est utilisée dans 90% des entreprises tech. À adopter dès maintenant.

Le README.md attendu

Le fichier README.md à la racine de votre repo doit contenir, dans l'ordre :

README.md

````
# Le Calao Doré — Restaurant Camerounais

> Site vitrine d'un restaurant traditionnel camerounais à Douala.
> Projet réalisé dans le cadre d'Angular Talent Lab 2026.

## Captures d'écran

### Desktop
![Desktop screenshot](./screenshots/desktop.png)

### Mobile
![Mobile screenshot](./screenshots/mobile.png)

## Technologies utilisées

- Angular 22.0.1
- Bootstrap 5.3
- TypeScript 5.9+
- npm 11.17.0

## Fonctionnalités

- ☑ Layout responsive (Desktop / Tablette / Mobile)
- ☑ Menu burger mobile
- ☑ Carte interactive des plats avec catégories
- ☑ Témoignages clients avec étoiles dynamiques
- ☑ Horaires avec indicateur du jour ouvert
- ☑ Footer multi-colonnes

## Lancer le projet en local

```bash
git clone https://github.com/[VOTRE_USERNAME]/le-calao-dore.git
cd le-calao-dore
npm install
ng serve --port 8080 --open
```

Ouvrir [http://localhost:8080](http://localhost:8080).

## Auteur
````

```
[VOTRE NOM] — Apprenant Angular Talent Lab 2026 — Cohorte Douala
```

Pour les captures d'écran : créez un dossier screenshots/ à la racine et placez-y vos images PNG.

Grille d'évaluation — 20 points

Voici comment je vais noter votre projet le lundi 22 juin matin. Familiarisez-vous avec cette grille AVANT de coder — c'est ce qui sera vérifié.

#	Critère évalué	Points	Acquis
1	Projet Angular 22 fonctionnel (ng serve marche)	**2 pts**	J1+J2
2	Bootstrap correctement installé (ng add)	**1 pt**	J4
3	Tous les composants sont Standalone avec imports[]	**2 pts**	J2
4	Hiérarchie de composants conforme (6 composants)	**1 pt**	J3
5	Header avec navbar + burger fonctionnel sur mobile	**2 pts**	J3+J4
6	Section Menu avec @for + @switch + ngClass	**3 pts**	J4
7	Section Témoignages avec @for imbriqué pour étoiles	**2 pts**	J4
8	Section Horaires avec jour actuel détecté	**2 pts**	J4
9	Responsive Desktop / Tablette / Mobile fonctionnel	**2 pts**	J3
10	Code Git propre (10+ commits clairs)	**1 pt**	J1
11	README.md complet avec captures	**1 pt**	—
12	Bonus : créativité, animations, finitions	**1 pt**	—

BARÈME DE NIVEAU

18-20 pts : EXCELLENT — vous êtes prêt(e) pour un projet en entreprise
15-17 pts : TRÈS BIEN — solide maîtrise des acquis, quelques détails à parfaire
12-14 pts : BIEN — bonne base, retravailler les points faibles avant J5
9-11 pts : MOYEN — séance de rattrapage prévue lundi après-midi
< 9 pts : INSUFFISANT — refaire le devoir en binôme avec un apprenant à 15+

Aide et ressources

Où chercher quand vous êtes bloqué(e)

16. Vos supports de cours des Jours 1 à 4 — c'est tout ce que vous devez savoir
17. La documentation officielle : angular.dev/guide
18. La documentation Bootstrap : getbootstrap.com/docs
19. Le WhatsApp de la cohorte — entraide encouragée pour DÉBUGGER, pas pour copier

Politique d'entraide pendant le week-end

- Discutez de la logique, des erreurs, des concepts — c'est sain et formateur.
- MAIS chaque apprenant écrit SON code, sur SON repo, avec SES commits.
- Copier-coller du code d'un autre apprenant = 0 sur le devoir, pour les deux.
- J'utiliserai un détecteur de similarité de code lundi matin.

Politique d'utilisation de l'IA (ChatGPT, Claude, Copilot)

- AUTORISÉ : utiliser l'IA pour comprendre une erreur, demander des explications.
- AUTORISÉ : générer du contenu textuel (noms de plats, descriptions, témoignages).
- INTERDIT : copier-coller du code généré par IA sans le comprendre.
- Si je vous interroge lundi sur n'importe quelle ligne de votre code, vous devez pouvoir l'expliquer.

BLOCAGE TECHNIQUE MAJEUR

Si vous êtes bloqué(e) plus de 1h30 sur la même erreur, contactez le formateur sur le WhatsApp privé (pas le groupe).

Indiquez : votre nom, la section sur laquelle vous bloquez, le message d'erreur exact, ce que vous avez essayé.

Je vous aiderai à débloquer la situation. Mais d'abord, essayez VRAIMENT seul(e).

Checklist finale avant le rendu

Dimanche soir à 18h00, AVANT de pousser sur GitHub, vérifiez TOUS ces points :

Fonctionnel

- ☐ Le site se lance avec ng serve --port 8080 sans erreur
- ☐ Aucune erreur rouge dans la console du navigateur (F12)
- ☐ Le menu burger s'ouvre et se ferme sur mobile
- ☐ Les 8 plats s'affichent avec leurs badges colorés
- ☐ Les 3 témoignages affichent le bon nombre d'étoiles
- ☐ Le jour actuel est mis en évidence dans les horaires

Responsive

- ☐ Vue Desktop ≥ 900px : navbar horizontale, menu 3 colonnes
- ☐ Vue Tablette 600-900px : menu 2 colonnes
- ☐ Vue Mobile < 600px : burger, menu 1 colonne, footer empilé
- ☐ Test fait sur un VRAI téléphone (pas que les DevTools)

Code

- ☐ Tous les composants sont Standalone (pas de NgModule)
- ☐ Chaque composant a son tableau imports[] correctement rempli
- ☐ Tous les @for ont la propriété track
- ☐ Aucun *ngIf ou *ngFor (ancienne syntaxe) dans le code
- ☐ Code indenté et lisible

Git & GitHub

- ☐ Le repo est PUBLIC sur votre GitHub
- ☐ Au moins 10 commits avec messages clairs
- ☐ Aucun fichier node_modules/ commit (vérifiez le .gitignore)
- ☐ ebangaarnaud@gmail.com invité comme collaborateur

README

- ☐ README.md à la racine avec votre nom
- ☐ Capture Desktop incluse
- ☐ Capture Mobile incluse
- ☐ Commande pour lancer le projet documentée

🚀 BONNE CHANCE !

Vous avez TOUT pour réussir : 4 jours de cours solides, un cadre détaillé, du temps suffisant.

Ce projet va devenir le premier élément de votre portfolio de développeur Angular.

Quand il sera fini et déployé, vous aurez un VRAI argument à montrer aux recruteurs locaux et internationaux.

Donnez-vous à fond ce week-end. C'est l'investissement le plus rentable de votre formation.

À lundi 9h00.